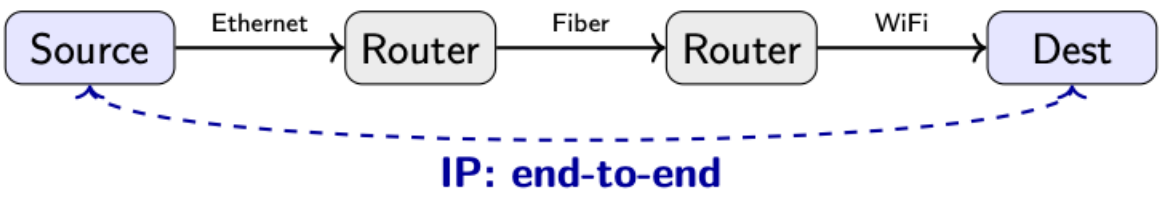
Lecture 13 - The Network Layer

Acronym	Meaning	Acronym	Meaning
ASIC	Application-Specific Integrated Circuit	LPM	Longest Prefix Match
BGP	Border Gateway Protocol	OSPF	Open Shortest Path First
DSCP	Differentiated Services Code Point	QoS	Quality of Service
FIB	Forwarding Information Base	RED	Random Early Detection
HOL	Head-of-Line (blocking)	RIB	Routing Information Base
ICMP	Internet Control Message Protocol	SDN	Software-Defined Networking
IP	Internet Protocol	SOHO	Small Office / Home Office
ISP	Internet Service Provider	TCAM	Ternary Content-Addressable Memory
IXP	Internet Exchange Point	TTL	Time to Live

From Links to End to End

- Physical layer and data link layer get frames between directly connected nodes one hop at a time
- The network layer solves the harder problem → deliver a packet from a source host to a destination host many hops away across many networks



Properties of IP:

- Best effort: no delivery guarantee / no ordering / no timing
- Connectionless: each packet forwarded independently
- Universal: runs on every device over any link technology

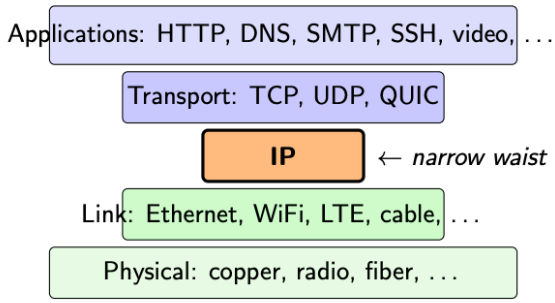
Network layer: three responsibilities

1. Addressing
    a. Give every host a globally unique identifier so we can name the destination
2. Forwaring
    a. At each router, look at incoming packets destination address and decide which output port to use
3. Routing
    a. Across the network, compute the paths that forwarding tables should encode

Also does:

1. Getting your IP address
2. Translating names to addresses

IP: the narrow waist : internet architecture is an hourglass → IP is the single protocol everything runs on top of → IPs simplicity is a feature, best effort and connectionless makes it trivial to implement any new link technology and lets TCP make its own choice



Why best effort?

- Best effort is a deliberate design choice

> **End-to-End Argument (informal)**
>
> If a function requires application-specific knowledge to implement correctly, it *must* be implemented at the endpoints. Doing it inside the network is redundant — the application still has to check anyway.

-
- Applied to reliability: even if every router verified delivery, your application still needs to confirm the data arrived correctly at the right process and was acted on. So implement reliability once in TCP and at the endpoints
- Where does reliability actually live?
    - Link layer (below IP) → CRC catches bit errors hop by hop
    - Transport layer (above IP) → TCP provides ordered reliable, end to end byte stream delivery
    - IP itself → does nothing and drops corrupt packets

What best effort does not provide: QoS Dimensions

Quality of Service (QoS) → guarantees an application might need from the network best effort IP provides none of these

Dimension	Meaning	IP best-effort	Where it's handled
Latency	End-to-end delay	Unbounded	CDNs; congestion control
Jitter	Delay *variation*	Unbounded	Jitter buffers (RTP, QUIC)
Packet loss	% packets dropped	Not guaranteed zero	TCP retransmit; app tolerates
Ordering	Packets arrive in sequence	Not guaranteed	TCP reorders; UDP does not
Throughput	Minimum guaranteed rate	None	TCP fair sharing; DiffServ

Jitter is often more important than latency

A VoIP call with 150 ms *constant* delay sounds fine. A call with 20–200 ms *varying* delay sounds broken. This is why voice/video uses UDP + application-level jitter buffers rather than TCP — TCP's retransmission makes jitter *worse*.

Best effort vs the alternatives

Two other options:

X.25

- Connection oriented / reliable packet delivery at the network layer
- Dominant in enterprise networks
- ITU standard; backbone of early internet
- Expensive per hop state, slow to deploy new features
- Largesly obsolete by mid 1990s

ATM

- Fixed 53 byte cells; hardware switched
- Quality of service guaranteed in the network
- Complex and expensive
- Lost to IP and faster ethernet

Good enough beats perfect

Why IP/ Ethernet won:

- Ran cheap commodity hardware
- Simple to implement
- Failures handled in software at endpoints
- Open standard

Why complex system lost:

- Per connection state in every switch is expensive
- Standard bodies moved slowly
- Guaranteees build for voice traffic didnt match data applications
- Once IP had networks effects, no migration path existed

The lasting lesson

A simple, "good enough" design that can be deployed and iterated on quickly will often beat a complex, "perfectly engineered" design that takes a decade to standardize. This pattern recurs throughout computing.

Two jobs, two timescales

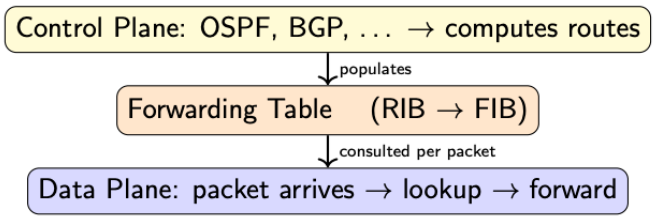
Every 2 routers has 2 fundamentally different functions:

Data plane → fast path

- Per packet processing
- Nano seconds to micro seconds
- Look at header, look up table, send
- Implemented in custom hardware
- Dumb but fast

Control plane → <u>slow path</u>

- Builds and maintains forwarding tables
- Seconds to minutes
- Runs routing protocols
- Runs on general purpose CPU
- Smart but slow



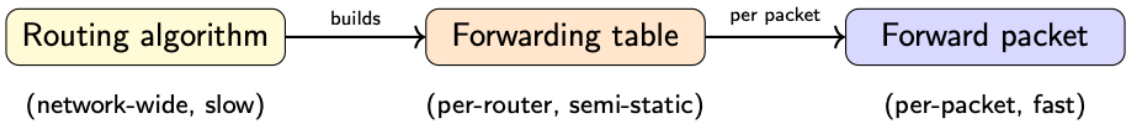
Reading routing table

Routing table is the interface between control plane and data plane

```
$ ip route show
default via 192.168.1.1 dev eth0 proto dhcp src 192.168.1.42 metric 100
10.0.0.0/8 via 10.1.0.1 dev eth1 proto static
192.168.1.0/24 dev eth0 proto kernel scope link src 192.168.1.42
```

- default = 0.0.0.0/0 — matches anything not more specifically matched
- 10.0.0.0/8 — 10.x.x.x range; installed by static config
- 192.168.1.0/24 scope link — local subnet; directly reachable

Forwarding vs routing



Forwarding:

- Given this packet, which output port?
- Local to a single router
- Data plane → fast lookup table

Routing:

- What should the tables say
- Network wide computation
- Control plane

What is a router → <u>a router is a device that forwards packets between networks at the IP layer (layer 3) and it connects networks not just hosts</u>

<u>→</u> in contrast, a switch forwards frames withing a single network

Where the router lives:

- Home → wifi router
- Bu campus → edge routers connects BU to the internet
- ISP network → aggregate traffic from thousands of customers

Router scale and location:

Type	Example	Speed
Home/SOHO	Netgear, TP-Link	1 Gb/s
Campus edge	Cisco ASR 1000	10–100 Gb/s
ISP edge	Cisco ASR 9000	100 Gb/s–1 Tb/s
Tier-1 core	Cisco CRS-X Juniper PTX	1–10+ Tb/s

Forwarding: step by step

What a router data plane does for each arriving packet:

1. Receive bits on input interface; reconstruct packet
2. Check IP header checksum; decrement TTL and update checksum
3. If TTL = 0: drop packet, send ICMP Time Exceeded to source
4. Look up destination IP in forwarding table using longest-prefix match
5. Send packet out the matching output interface (ARP for next-hop MAC if needed)

TTL: Loop Prevention

Time to Live is decremented at each hop. A packet that gets stuck in a routing loop is eventually dropped rather than circulating forever. Typical initial values: 64 (Linux), 128 (Windows), 255 (Cisco).

Traceroute → Sends packets with TTL = 1, 2, 3, . . . Each router that drops a TTL-expired packet reveals itself via ICMP Time Exceeded

The internet is smaller than you think

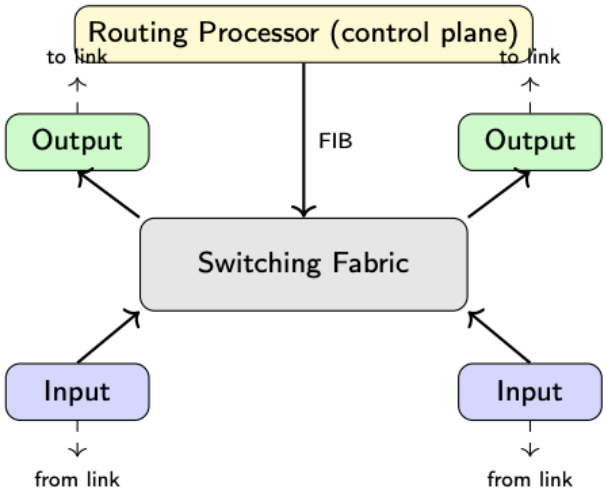
Typical trace route to any destination worldwide: <u>10-15 hops</u>

TTL design implication

TTL = 20 would reach virtually anywhere in the internet. Linux default (64) is ∼4× larger than needed — a deliberate safety margin for unexpected routing loops or long paths. Windows (128) and Cisco (255) are even more conservative.

Router architecture

- iinput ports
  - Physical → receive bits and clock recovery
  - Link layer → strip header/ check FCS
  - Lookup destination IP → output port
- Switching fabirc
  - Interconnects input and output ports
  - Must run faster than sum of all input rates
- Output ports:
  - Buffer packets from switching fabric
  - Add link-layer header; transmit
- Routing processor (control plane):
  - Runs OSPF, BGP
  - Builds RIB → installs FIB into line cards



Queuing, packet loss, and HOL blocking

Queues form at output ports when packets arrive from the fabric faster than the link ca drain them

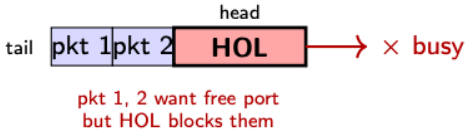
→ when queue overflows → packets are dropped

<u>Drop policies:</u>

- Tail drop → drop the nearest packet when queue is full
- RED (random early detection) → drop probabilisitcally before queue is full to signal TCP senders early

<u>Head of Line (HOL) blocking</u> → a packet at the head of an input queue waiting for a busy output port blocks packrets behind it that could go to a free port

→ fix by <u>Virtual Output Queue (VOQ)</u> → separate queue per output at input



Longest prefix match (LPM) → routing tables store prefixes that aggregate many addresses. A destination IP can match multiple entries of different lengths

Rule: Always use the most specific (longest) matching prefix

The longer the prefix, the more specific the routing instruction.

Prefix	Prefix Length	Next Hop	Interface	
0.0.0.0/0	0	203.0.113.1	eth0	(default)
10.0.0.0/8	8	10.255.0.1	eth1	
10.1.0.0/16	16	10.1.255.1	eth2	
10.1.2.0/24	24	10.1.2.254	eth3	

- Destination 10.1.2.5 → matches /0, /8, /16, /24 → **use /24** (eth3)
- Destination 10.5.6.7 → matches /0, /8 → **use /8** (eth1)
- Destination 172.16.0.1 → matches /0 only → **default route** (eth0)

Prefix match is bitwise not byte aligned

Prefix length need not be a multiple of 8. The mask boundary can fall *anywhere* within a byte.

Example: 192.168.32.0/20 — the boundary falls at bit 20, mid-way through byte 3:

```
Network: 11000000.10101000.0010|0000.00000000 (192.168.32.0)
Mask /20: 11111111.11111111.1111|0000.00000000
                          ^-- bit 20 boundary
```

192.168.33.5 **MATCH**

```
11000000.10101000.0010|0001.00000101
                    ^ same 20 bits
```

192.168.48.1 **NO MATCH**

```
11000000.10101000.0011|0000.00000001
                    ^^ bits 17-20 differ
```

Hardware LPM: TCAM

Problem:

- Sequential seach through so many routing tables entries is impossible

Solution: TCAM (ternary content addressable memory)

- Stores each entry as a (value, mask) pair
- Ternary: each bit can bs 0 , 1 , or x (dont care) for IP prefix
- Seach all entries in parallel in a single clock
- Returns longest matching entry

Tradeoff:

- TCAM is expensive in power and chip area

Rethinking the control plane

<u>Traditional model:</u> control plane is distributed → OSPF routers exchange link state advertisements → each runs dijkstras independently

<u>Fundamental limitation:</u> each router only has a local view. Hard to enforce network wide policies or optimize global traffic

The SDN Insight

If the data plane and control plane are cleanly separated, the control plane does not need to run *inside* the router. It can run anywhere — on servers with a **global view** of the entire network.

Traditonal

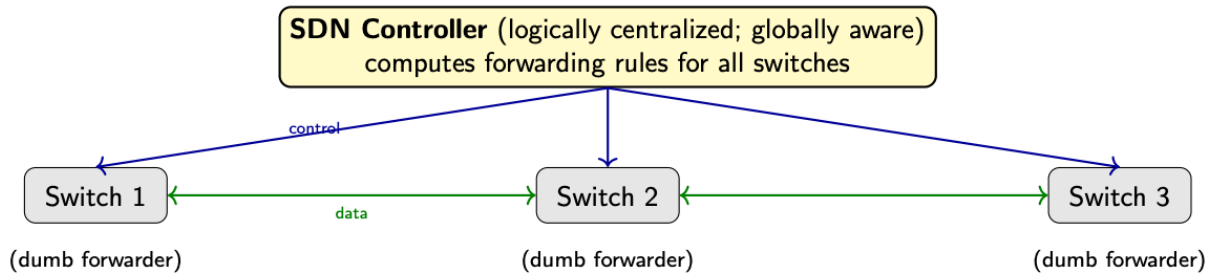
- Control plane embedded in every router

- Distributed routing protocols
- Policy expressed in complex configs

SDN

- Centralized controller has global topology view
- Pushes forwarding rules ton ‘dumb’ switches
- Policy expressed in software

SDN architecture



OpenFlow: controller pushes flow table rules to switches (“if dst IP matches `10.1.2.0/24`, send out port 3”). Switches just match and act.

SDN today: cloud vs open internet

Where SDN IS the present → cloud data centers:

- Google B4: SDN-controlled WAN between data centers; near-100% link utilization vs. 30–40% with traditional OSPF
- AWS VPC: networking inside AWS is SDN — controller pushes rules to hypervisors; no traditional routing between VMs
- Azure, Meta: similar fabric architectures

Key insight: SDN works where one
organization controls all the infrastructure